

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN



IMPLEMENTACIÓN DE OPERACIONES CON ARREGLOS EN JAVA
--

ALUMNO: VEGA MUÑOZ JHONSTYN YERAY

DOCENTE: ING. FERNANDEZ BEJARANO RAUL ENRIQUE

CICLO: IV

CODIGO: U01024A

SECCIÓN: A1

**HUANCAYO - PERÚ
2026**

INTRODUCCIÓN

La presente actividad desarrolla una aplicación en Java utilizando Apache NetBeans para trabajar con arreglos de números enteros. La solución se organiza en dos clases: Arreglo, que concentra los datos y las operaciones sobre el arreglo, y AplicaArreglo, que contiene el método main () y un menú desde el cual el usuario ejecuta cada operación. Esta separación permite mantener la lógica del arreglo independiente de la interacción con el usuario y facilita la lectura, prueba y mantenimiento del programa.

La actividad comprende la eliminación, modificación e inserción de elementos; el cálculo de sumas, menor, mayor y promedio; y el desplazamiento de elementos hacia la última posición. Para evidenciar el desarrollo se incluyen el enunciado, el TDA trabajado en clase, la estructura del proyecto, las capturas del código y las pruebas de ejecución realizadas.

ENUNCIADO DE LA ACTIVIDAD

La Actividad 1 solicita implementar ocho métodos sobre un arreglo creado previamente. Cada método resuelve una operación específica y luego es llamado desde el menú principal de la clase AplicaArreglo.

ACTIVIDAD 1

1. Elaborar un método para eliminar un elemento "x", del arreglo creado anteriormente.
2. Elaborar un método para modificar un elemento "x".
3. Elaborar un método para insertar un elemento "x" en una posición "y"
4. Elaborar un método para sumar los elementos del arreglo.
5. Elaborar un método para hallar el número menor, el número mayor y el promedio de los elementos del arreglo.
6. Elaborar un método para suma de Impares los elementos del arreglo.
7. Elaborar un método para mover el Primer elemento a la Última posición del arreglo.
8. Elaborar un método para mover el Primer elemento Par al Final del arreglo.

Los ocho requerimientos son: eliminar un elemento x; modificar un elemento x; insertar un elemento x en una posición y; sumar todos los elementos; hallar el menor, el mayor y el promedio; sumar los elementos impares; mover el primer elemento a la última posición; y mover el primer elemento par al final del arreglo.

OBJETIVOS

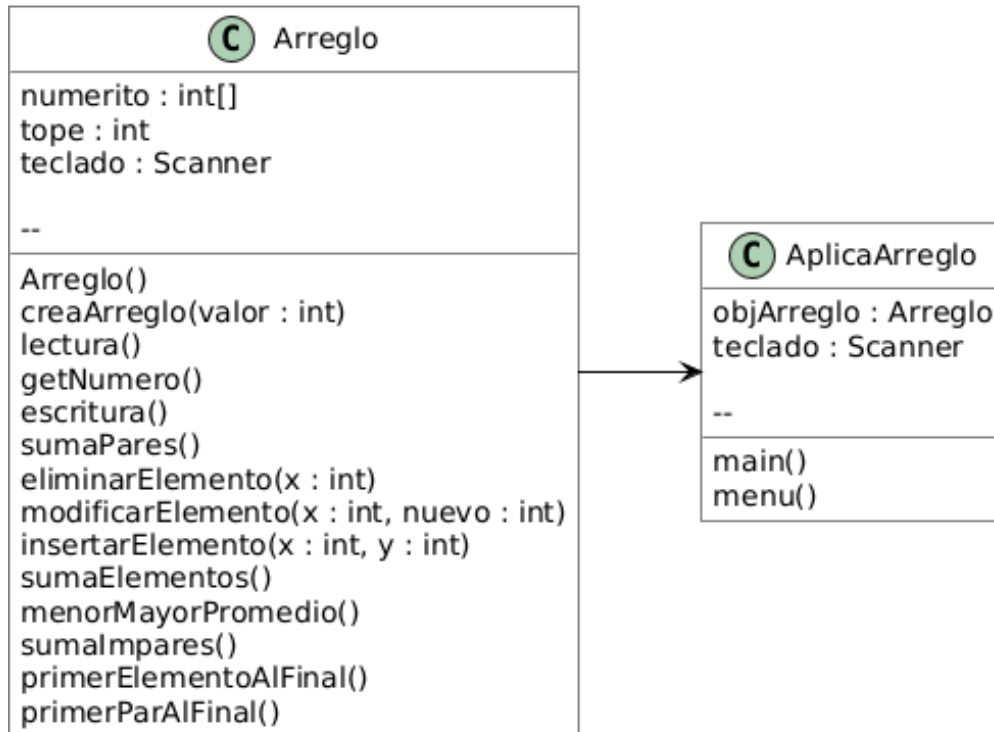
Objetivo general. Implementar operaciones sobre arreglos de enteros en Java mediante métodos independientes, utilizando estructuras condicionales, ciclos y una organización básica orientada a objetos.

Objetivos específicos. Aplicar recorridos lineales para localizar elementos; modificar la estructura lógica del arreglo mediante desplazamientos; efectuar cálculos acumulativos y estadísticos; y comprobar cada operación mediante un menú interactivo y pruebas de ejecución.

TDA Y ORGANIZACIÓN DE CLASES

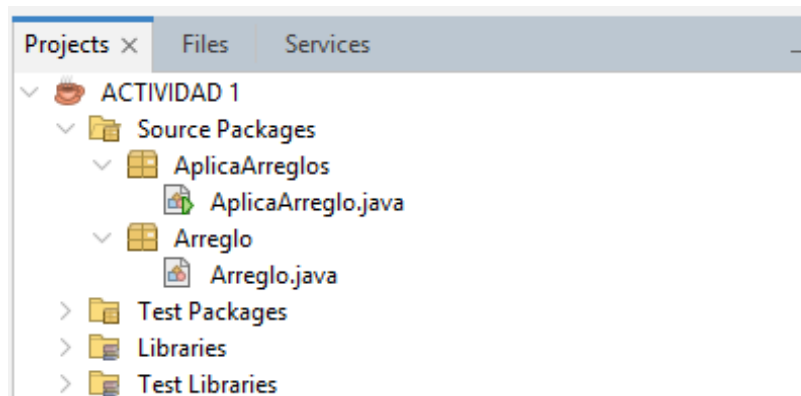
El TDA fue planteado en clase con dos clases relacionadas. La clase Arreglo administra el arreglo numerito, la variable tope y los métodos que operan sobre los datos. La clase AplicaArreglo crea un objeto de Arreglo y presenta el menú que permite llamar a dichos métodos.

TDA - ACTIVIDAD 1



ESTRUCTURA DEL PROYECTO EN APACHE NETBEANS

El proyecto se organiza en dos paquetes separados. **AplicaArreglos** contiene la clase `AplicaArreglo.java`, responsable de la ejecución y del menú. **Arreglo** contiene la clase `Arreglo.java`, donde se encuentran el arreglo y todos los métodos de procesamiento. Esta separación permite que la clase principal utilice la lógica de **Arreglo** mediante el objeto `objArreglo`.



EJERCICIOS

1) Eliminación de un elemento:

Definición:

La eliminación de un elemento consiste en retirar un valor determinado que se encuentra almacenado dentro del arreglo. Como los arreglos mantienen sus datos en posiciones consecutivas, no es posible simplemente borrar una posición, por ello se realiza un proceso de reorganización. Los elementos ubicados después del dato eliminado se desplazan una posición hacia la izquierda para mantener la continuidad de la información y finalmente la variable tope se actualiza para representar la nueva cantidad de elementos utilizados.

Algoritmo:

```
// Eliminar elemento
public void eliminaElemento(int valor) {
    for (int r = 0; r < getTope(); r++) {
        if (getNumerito()[r] == valor) {
            for (int j = r; j < getTope() - 1; j++) {
                getNumerito()[j] = getNumerito()[j + 1];
            }
            setTope(getTope() - 1);
            break;
        }
    }
}
```

Explicación:

El algoritmo inicia realizando un recorrido secuencial mediante un ciclo for, analizando solamente las posiciones que contienen datos válidos, determinadas por la variable tope. De esta manera se evita revisar espacios que todavía no forman parte del arreglo utilizado.

for (int r = 0; r < getTope(); r++)

Durante el recorrido se utiliza una estructura condicional if para comparar cada elemento del arreglo con el valor recibido como parámetro. Esta comparación permite identificar la posición exacta donde se encuentra el dato que será eliminado.

if (getNumerito()[r] == valor)

Una vez encontrada la posición del elemento, se ejecuta un segundo recorrido que mueve los valores posteriores una posición hacia la izquierda. Este desplazamiento permite conservar el orden de los datos y evita generar posiciones vacías dentro del arreglo.

```
for (int j = r; j < getTope() - 1; j++) {
    getNumerito()[j] = getNumerito()[j + 1];
}
```

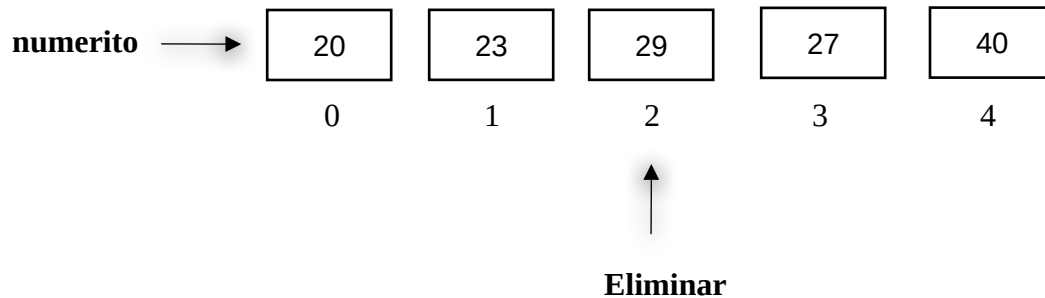
Finalmente, se actualiza el valor de tope para indicar que el arreglo ahora contiene un elemento menos. El comando break finaliza la búsqueda después de realizar la primera eliminación.

```
setTope(getTope() - 1);
```

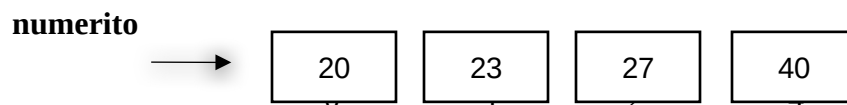
El break permite terminar la búsqueda después de eliminar el primer elemento encontrado.

Ejemplo gráfico:

Supongamos que tenemos:



Se desplazan los elementos posteriores:



Y tope disminuye de 5 a 4.

2) Modificación de un elemento:

Definición:

La modificación de un elemento permite cambiar el contenido de un dato ya almacenado dentro del arreglo sin afectar la distribución de las demás posiciones. El algoritmo localiza el valor indicado y lo reemplaza por un nuevo dato manteniendo la misma ubicación.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Modificar elemento
public void modificaElemento(int valor, int nuevoValor) {
    for (int r = 0; r < getTope(); r++) {
        if (getNumero(r) == valor) {
            getNumero(r) = nuevoValor;
            break;
        }
    }
}
```

Explicación:

Para realizar la modificación se ejecuta un recorrido desde la primera posición hasta el último elemento utilizado. En cada iteración se analiza el contenido de la posición actual hasta encontrar el valor solicitado.

```
for (int r = 0; r < getTope(); r++)
```

La condición if permite comprobar si el valor actual coincide con el elemento que se desea modificar.

```
if (getNumero(r) == valor)
```

Cuando existe coincidencia, el contenido de esa posición es reemplazado por el nuevo valor indicado.

```
getNumero(r) = nuevoValor;
```

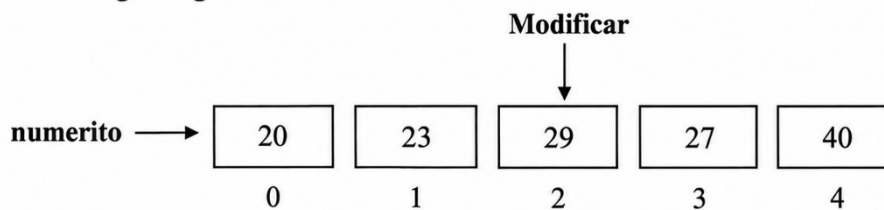
Finalmente, utilizamos:

```
break;
```

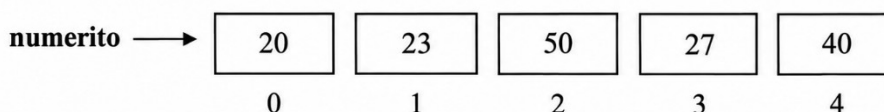
El proceso termina después de modificar el primer elemento encontrado dentro del arreglo.

Ejemplo gráfico:

En el arreglo original tenemos:



Después de la modificación, el arreglo queda así:



El número 29 se reemplaza por 50 y los demás elementos permanecen en la misma posición.

3) Inserción de un elemento:

Definición:

La inserción permite incorporar un nuevo elemento dentro del arreglo en una ubicación determinada. Debido a que los datos existentes deben conservarse, primero se liberará espacio desplazando los elementos hacia la derecha y posteriormente se almacenará el nuevo valor en la posición indicada.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Insertar elemento
public void insertaElemento(int valor, int posicion) {
    if (getTope() < getNumerito().length) {
        for (int r = getTope(); r > posicion; r--) {
            getNumerito()[r] = getNumerito()[r - 1];
        }
        getNumerito()[posicion] = valor;
        setTope(getTope() + 1);
    }
}
```

Explicación:

Antes de insertar se verifica que el arreglo tenga capacidad disponible para almacenar un nuevo dato.

if (getTope() < getNumerito().length)

Esto permite comprobar que el arreglo no esté lleno.

Los elementos se desplazan desde la última posición ocupada hacia la derecha para evitar perder información almacenada.

```
for (int r = getTope(); r > posicion; r--) {
    getNumerito()[r] = getNumerito()[r - 1];
}
```

El desplazamiento se realiza **desde el final hacia la posición indicada**, para evitar perder los valores que ya se encuentran en el arreglo.

Después del desplazamiento, el nuevo valor se asigna en la posición indicada por el usuario.

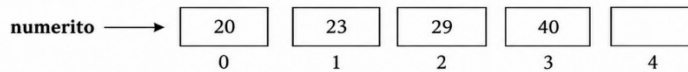
getNumerito()[posicion] = valor;

Finalmente, aumentamos tope en uno porque ahora tenemos un elemento más:

setTope(getTope() + 1);

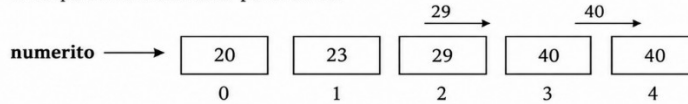
Ejemplo gráfico:

Supongamos que tenemos:

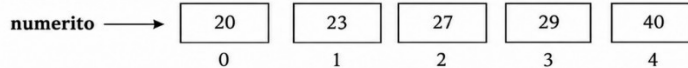


Insertar 27 en posición 2

Se desplazan los elementos posteriores:



Resultado final:



Los elementos desde la posición indicada se desplazan una posición a la derecha y se coloca el nuevo elemento.

4) Suma de todos los elementos:

Definición:

La operación de suma permite calcular el total acumulado de todos los valores almacenados en el arreglo. Para ello se emplea una variable acumuladora que inicia en cero y va incorporando cada elemento durante el recorrido hasta obtener el resultado final.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Sumar elementos
public void sumaElementos() {
    int suma = 0;

    for (int r = 0; r < getTope(); r++) {
        suma = suma + getNumerito()[r];
    }

    System.out.println("La Suma de los elementos es : " + suma);
}
```

Explicación:

Se inicializa la variable suma con cero porque será utilizada para almacenar el resultado acumulado.

```
int suma = 0;
```

Esta variable será la encargada de almacenar el resultado de la suma.

Mediante un ciclo for se recorren todas las posiciones que contienen información válida dentro del arreglo.

```
for (int r = 0; r < getTope(); r++)
```

El ciclo comienza desde la posición 0 y continúa hasta la última posición utilizada del arreglo.

En cada repetición, el elemento actual se agrega a la variable suma:

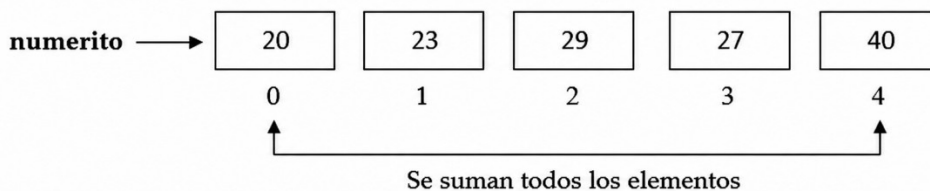
Cada elemento encontrado se agrega al acumulador hasta obtener la suma completa.

Finalmente, mostramos el resultado mediante:

```
System.out.println("La Suma de los elementos es : " + suma);
```

Ejemplo gráfico:

Supongamos que tenemos:



El cálculo es:

$$\boxed{20} + \boxed{23} + \boxed{29} + \boxed{27} + \boxed{40} = \boxed{139}$$

La suma de todos los elementos es 139.

5) Cálculo del menor, mayor y promedio:

Definición:

Este método realiza un análisis completo del contenido del arreglo para obtener tres resultados importantes: el valor mínimo, el valor máximo y el promedio matemático de todos los elementos registrados. Para ello se realizan comparaciones y una acumulación de valores durante un único recorrido.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Menor mayor promedio
public void menorMayorPromedio() {

    int menor = getNumerito()[0];
    int mayor = getNumerito()[0];
    int suma = 0;
    float promedio;

    for (int r = 0; r < getTope(); r++) {

        if (getNumerito()[r] < menor) {
            menor = getNumerito()[r];
        }

        if (getNumerito()[r] > mayor) {
            mayor = getNumerito()[r];
        }

        suma = suma + getNumerito()[r];
    }

    promedio = (float) suma / getTope();

    System.out.println("El numero menor es : " + menor);
    System.out.println("El numero mayor es : " + mayor);
    System.out.println("El promedio es : " + promedio);
}
}
```

Explicación:

Las variables menor y mayor toman inicialmente el primer valor del arreglo como referencia para realizar las comparaciones.

```
int menor = getNumerito()[0];
```

```
int mayor = getNumerito()[0];
```

Esto permite utilizar el primer elemento como referencia para realizar las comparaciones.

También declaramos una variable para acumular la suma:

```
int suma = 0;
```

y una variable float para almacenar el promedio:

```
float promedio;
```

Durante el recorrido se comparan los elementos para actualizar los valores de menor y mayor cuando corresponde.

```
for (int r = 0; r < getTope(); r++)
```

Para encontrar el menor, comparamos cada elemento con la variable menor:

```
if (getNumerito()[r] < menor) {
    menor = getNumerito()[r];
}
```

Si encontramos un número más pequeño, este pasa a convertirse en el nuevo valor de menor.

De manera similar, buscamos el número mayor:

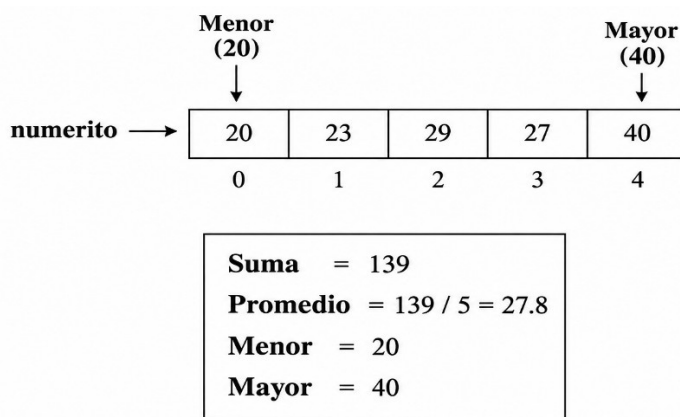
```
if (getNumerito()[r] > mayor) {  
    mayor = getNumerito()[r];  
}
```

Durante el mismo recorrido también acumulamos todos los elementos:

```
suma = suma + getNumerito()[r];  
promedio = (float) suma / getTope();
```

La suma se divide entre la cantidad de elementos que contiene el arreglo.

Ejemplo gráfico:



6) Suma de elementos impares:

Definición:

La suma de elementos impares consiste en seleccionar únicamente aquellos valores que no son divisibles entre dos y obtener su suma total. Para identificar estos elementos se utiliza el operador módulo (%), que permite conocer el residuo de una división.

Este proceso consiste en **recorrer el arreglo, identificar cuáles elementos son impares y sumarlos**. Para saber si un número es impar, utilizamos el operador %. Si al dividir el número entre 2 el residuo es diferente de 0, significa que el número es impar.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Suma de impares
public void sumaImpares() {

    int suma = 0;

    for (int r = 0; r < getTope(); r++) {

        if (getNumerito()[r] % 2 != 0) {
            suma = suma + getNumerito()[r];
        }
    }

    System.out.println("La Suma de los impares es : " + suma);
}
```

Explicación:

La variable suma comienza en cero y almacenará únicamente los elementos que cumplan la condición de imparidad.

int suma = 0;

Esta variable almacenará la suma de los números impares encontrados.

Luego recorreremos los elementos utilizados del arreglo:

for (int r = 0; r < getTope(); r++)

El operador módulo (%) permite conocer el residuo de una división. Si el residuo al dividir entre 2 es diferente de cero, el número es impar.

if (getNumerito()[r] % 2 != 0)

El operador % obtiene el residuo de una división. Cuando un número impar se divide entre 2, el residuo es 1, por lo que cumple la condición != 0.

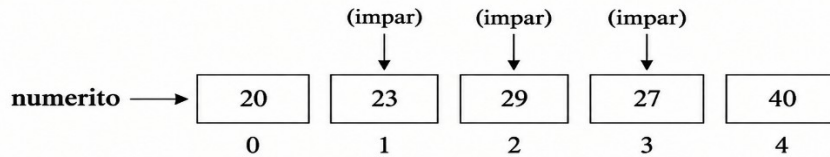
Si el número es impar, se agrega a la variable suma:

suma = suma + getNumerito()[r];

Finalmente, mostramos el resultado:

System.out.println("La Suma de los impares es : " + suma);

Ejemplo gráfico:



Se suman solo los impares.

$$23 + 29 + 27 = 79$$

La suma de los impares es 79.

7) Movimiento del primer elemento al final:

Definición:

Esta operación modifica la distribución del arreglo trasladando el primer elemento hasta la última posición disponible. Para realizarlo correctamente se almacena temporalmente el primer valor en una variable auxiliar y luego se desplazan los demás elementos hacia la izquierda conservando su orden.

Algoritmo:

Este es el método que tenemos en nuestra nueva clase Arreglos.java:

```
// Primer elemento al final
public void primerElementoAlFinal() {

    if (getTope() > 1) {

        int aux = getNumerito()[0];

        for (int r = 0; r < getTope() - 1; r++) {
            getNumerito()[r] = getNumerito()[r + 1];
        }

        getNumerito()[getTope() - 1] = aux;
    }
}
```

Explicación:

Primero se comprueba que existan suficientes elementos para realizar el movimiento.

```
if (getTope() > 1)
```

Esto permite realizar el movimiento cuando existen al menos dos elementos.

Después guardamos el primer elemento en una variable auxiliar:

El primer valor se guarda temporalmente en una variable auxiliar para conservarlo durante el desplazamiento.

La variable aux sirve para **no perder el valor del primer elemento** mientras realizamos los desplazamientos.

Luego recorreremos el arreglo y desplazamos cada elemento una posición hacia la izquierda:

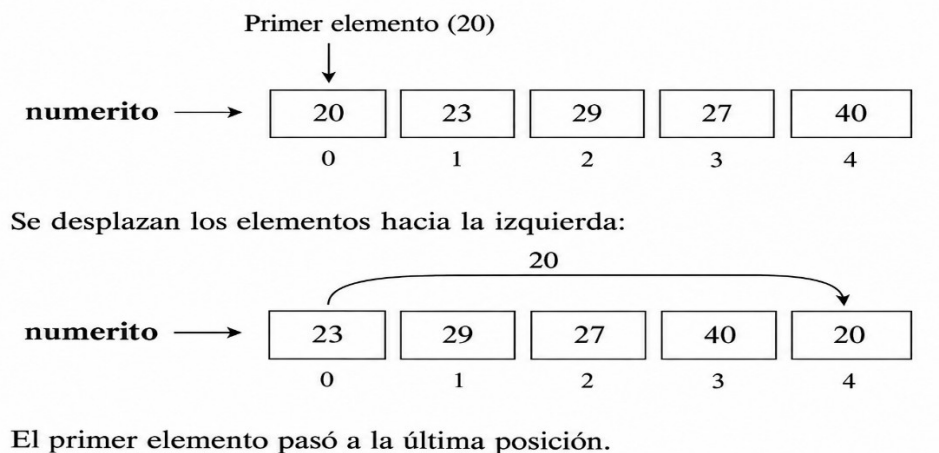
Los elementos restantes avanzan una posición hacia la izquierda y finalmente el valor almacenado se coloca al final del arreglo.

```
    getNumerito()[r] = getNumerito()[r + 1];  
}
```

Finalmente, colocamos el elemento que habíamos guardado en la última posición:

```
getNumerito()[getTope() - 1] = aux;
```

Ejemplo gráfico:



8) Movimiento del primer elemento par al final:

Definición:

Esta operación permite localizar únicamente el primer elemento par del arreglo y enviarlo hacia la última posición. Una vez identificado el valor, se guarda temporalmente, se reorganizan los elementos posteriores mediante desplazamientos y finalmente se coloca el número par al final.

Algoritmo:



```

// Primer elemento par al final
public void primerParAlFinal() {

    for (int r = 0; r < getTope(); r++) {

        if (getNumerito()[r] % 2 == 0) {

            int aux = getNumerito()[r];

            for (int j = r; j < getTope() - 1; j++) {
                getNumerito()[j] = getNumerito()[j + 1];
            }

            getNumerito()[getTope() - 1] = aux;
            break;
        }
    }
}

```

Explicación:

Se recorren las posiciones utilizadas del arreglo hasta encontrar el primer valor que cumpla la condición de ser par.

```
for (int r = 0; r < getTope(); r++)
```

Luego verificamos si el elemento actual es par:

La condición módulo permite comprobar si el número es divisible entre dos.

Si el residuo de dividir el número entre 2 es 0, significa que es un número par.

Cuando se encuentra el primer número par, se almacena temporalmente para poder reorganizar las posiciones del arreglo.

```
int aux = getNumerito()[r];
```

Después desplazamos hacia la izquierda los elementos que se encuentran después del número par:

Los elementos posteriores se desplazan una posición hacia la izquierda y el valor guardado se coloca en la última posición.

```
    getNumerito()[j] = getNumerito()[j + 1];
}
```

Una vez realizado el desplazamiento, colocamos el número par guardado en aux en la última posición:

```
getNumerito()[getTope() - 1] = aux;
```

Finalmente utilizamos:

El break detiene el recorrido porque solamente se requiere mover el primer elemento par encontrado.

para detener el recorrido, ya que **solo debemos mover el primer elemento par**, no todos los elementos pares.

Ejemplo gráfico:

